

GPU Architecture

CPU와 GPU의 구조적 차이

CPU는 복잡한 하나의 일을 빠르게, GPU는 단순한 수많은 일을 동시에 처리한다.

설계 철학 비교

항목	CPU	GPU
설계 목적	범용 처리, 복잡한 제어 흐름	대량 병렬 연산
코어 구조	소수의 강력한 코어	다수의 단순한 연산 유닛
회로 비중	제어 로직(캐시, 분기예측, 스케줄링) 중심	계산 유닛 중심
최적화 방향	Latency (응답 속도)	Throughput (총 처리량)
메모리 전략	지연시간을 줄임 (L1/L2/L3 캐시)	지연시간을 병렬성으로 숨김
분기 처리	분기 예측기로 빠르게 처리	Branch divergence 발생 시 성능 저하
스레드 특성	무거움, OS 관리, 수십 개 수준	가벼움, 수천~수만 개 동시 실행

코어 수의 의미

CPU 8코어와 GPU 4096코어는 단순 비교 불가. CPU 코어는 fetch/decode/branch prediction/cache/execuion을 독립 수행하는 완전한 처리 단위이고, GPU 코어는 같은 명령을 여러 데이터에 적용하는 작은 산술 유닛에 가까움.

실행 모델

- **CPU:** 독립적 명령 흐름을 빠르게 처리. Out-of-Order Execution, 파이프라인, 분기 예측 활용.
- **GPU:** SIMT 방식. 32개 스레드 묶음(NVIDIA: warp, AMD: wavefront)이 같은 명령을 공유하며 각자 다른 데이터 처리. 분기가 같으면 경로를 순차 실행하면서 일부 스레드가 놀게 됨.

적합한 작업

CPU에 강한 작업	GPU에 강한 작업
OS 실행, 프로그램 제어	행렬 곱셈, 딥러닝 학습/추론
DB 트랜잭션, 웹 서버	그래픽 렌더링, 이미지/영상 처리
컴파일, 조건 분기 많은 알고리즘	과학/물리 시뮬레이션, 암호 연산
파일/네트워크 I/O	같은 연산을 수백만 데이터에 반복 적용

CPU-GPU 데이터 이동

CPU(시스템 메모리)와 외장 GPU(VRAM)는 별도 메모리 공간을 사용하며, PCIe를 통해 데이터를 복사함. 이 전송 오버헤드가 연산 자체보다 클 수 있으므로, GPU가 유리하려면 **작업량이 충분히 크고, 병렬화가 잘 되고, 데이터 왕복이 적어야 함**.

기타 차이점

항목	CPU	GPU
캐시 일관성	MESI 등 하드웨어가 자동 관리	프로그래머가 동기화/barrier 명시 필요
전력/발열	클럭 조절, 코어 파킹으로 유연 대응	고부하 시 수백 W, 냉각 중요
통합 vs 외장	—	통합: 메모리 공유로 전송 비용 감소. 외장: 자체 VRAM으로 고성능

GPU 메모리 계층 구조

GPU 메모리는 속도-용량 트레이드오프에 따라 레지스터 → 공유 메모리 → L1/L2 캐시 → 글로벌 메모리 (VRAM) 순으로 계층화되어 있다.

메모리 계층 분류

계층	위치	용량	지연시간	접근 범위	특징
레지스터	SM 내부	스레드당 수십~수백 개	~1 cycle	스레드 전용	가장 빠름, 용량 제한 시 occupancy 감소
공유 메모리 (Shared Memory)	SM 내부	SM당 수십~수백 KB	~5~30 cycles	같은 블록 내 스레드 공유	프로그래머가 명시적 관리, 블록 간 협업 핵심
L1 캐시	SM 내부	SM당 수십~수백 KB	~30 cycles	SM 내부 자동 관리	공유 메모리와 물리적 공간을 분할하는 경우 다
L2 캐시	GPU 전체 공유	수 MB	~200 cycles	전체 SM 공유	글로벌 메모리 접근 전 마지막 버퍼
글로벌 메모리 (VRAM)	GPU 외부 (HBM/GDDR)	수~수십 GB	~400~800 cycles	모든 스레드 접근 가능	대역폭 높지만 지연시간 큼
상수 메모리 (Constant Memory)	전용 캐시 + VRAM	64 KB (CUDA 기준)	캐시 히트 시 ~1 cycle	모든 스레드 읽기 전용	전체 스레드가 같은 값 읽을 때 최적
텍스처 메모리	전용 캐시 + VRAM	캐시 크기 제한	캐시 히트 시 낮음	모든 스레드 읽기 전용	2D 공간 지역성 최적화, 필터링 하드웨어 내장

레지스터

GPU 레지스터 파일은 전체 크기가 매우 큼 (SM당 수만~수십만 개). 이는 수천 개 스레드가 각자 자기 레지스터를 유지한 채 빠르게 전환되어야 하기 때문. 단, 스레드당 레지스터 사용량이 많아지면 동시 활성 스레드 수 (occupancy)가 줄어들어 메모리 지연 은닉 능력이 떨어짐.

공유 메모리 (Shared Memory)

같은 스레드 블록 내 스레드들이 데이터를 주고받을 수 있는 고속 온칩 메모리. 글로벌 메모리에서 반복 접근할 데이터를 미리 적재(tiling)하여 대역폭 병목을 해소하는 용도로 핵심적. 프로그래머가 직접 할당/관리하므로

"소프트웨어 관리 캐시"라고도 부름.

주의점:

- Bank conflict: 여러 스레드가 같은 bank에 동시 접근하면 직렬화 발생
- 블록 간에는 공유 불가 — 블록 간 통신은 글로벌 메모리 + 동기화 필요

L1 캐시와 공유 메모리의 관계

아키텍처	관계
Kepler/Maxwell	L1과 공유 메모리가 같은 물리 SRAM, 비율 설정 가능 (예: 48KB shared + 16KB L1)
Volta 이후	통합 데이터 캐시로 합쳐짐, 하드웨어가 동적 분배

L1 캐시는 하드웨어가 자동 관리하고, 공유 메모리는 프로그래머가 명시적으로 관리한다는 점이 핵심 차이.

글로벌 메모리 (VRAM)

GPU의 주 메모리. HBM(High Bandwidth Memory) 또는 GDDR을 사용. 용량은 크지만 지연시간이 수백 사이클로 높음. GPU가 이 지연을 견딜 수 있는 이유는 수천 스레드를 전환하며 대기 시간을 숨기기 때문.

성능 핵심 — Coalesced Access:

같은 warp 내 스레드들이 연속된 메모리 주소에 접근하면 하나의 트랜잭션으로 합쳐짐(coalescing). 반대로 불규칙 접근(scattered access)은 다수 트랜잭션으로 분리되어 대역폭 낭비.

접근 패턴	효율
Coalesced (연속 주소)	최적 — 1회 트랜잭션
Strided (일정 간격)	간격에 따라 효율 저하
Random (불규칙)	최악 — 다수 트랜잭션

상수 메모리 & 텍스처 메모리

- **상수 메모리:** 모든 스레드가 동일한 값을 읽을 때 최적. 전용 캐시를 통해 브로드캐스트됨. 다른 주소를 읽으면 직렬화.
- **텍스처 메모리:** 2D/3D 공간 지역성에 최적화된 캐시. 이미지 처리, 보간(interpolation)이 필요한 경우 하드웨어 가속.

메모리 계층별 최적화 원칙

원칙	설명
데이터 재사용 → 공유 메모리에 적재	글로벌 메모리 반복 접근 제거 (tiling)
Coalesced access 유지	warp 내 스레드가 연속 주소 접근하도록 데이터 배치
레지스터 압박 관리	스레드당 레지스터 과다 사용 → occupancy 하락 → 지연 은닉 실패
Bank conflict 회피	공유 메모리 접근 시 패딩 등으로 bank 충돌 방지
글로벌 메모리 접근 최소화	연산 대비 메모리 접근 비율(arithmetic intensity)을 높이는 방향으로 설계

CPU 메모리 계층과의 차이

항목	CPU	GPU
캐시 목적	지연시간 최소화	대역폭 효율 극대화
캐시 관리	하드웨어 자동 (투명)	L1/L2는 자동, 공유 메모리는 수동
지연 대응	캐시 히트로 지연 제거	스레드 전환으로 지연 은닉
일관성	하드웨어 캐시 일관성 프로토콜	명시적 동기화/fence 필요
대역폭	수십 GB/s	수백~수천 GB/s (HBM)



GPU 메모리 최적화의 핵심은 "느린 메모리 접근을 줄이고, 빠른 메모리를 최대한 활용하며, 접근 패턴을 규칙적으로 유지하는 것"이다. 레지스터와 공유 메모리를 적극 활용하고, 글로벌 메모리는 coalesced access로 접근하며, 충분한 스레드를 유지해 지연을 숨기는 것이 성능의 기본이다.

GPU 실행 단위와 동작 흐름

CPU가 커널을 발행 하면, GPU가 블록을 SM에 분배 하고, SM 내 warp 스케줄러 가 매 사이클 실행 가능한 warp를 선택하여 명령어를 발행 하는 흐름

전체 동작 흐름 (단계별)

단계	위치	동작
1. 커널 발행	CPU → GPU	CPU가 커널 런치 명령을 GPU에 전달 (grid 크기, 블록 크기, 인자)
2. 블록 분배	GigaThread Engine	스레드 블록들을 가용 SM에 분배. 자원(레지스터, 공유메모리) 확인 후 배정
3. Warp 생성	SM 내부	배정된 블록을 32스레드 단위 warp로 분할, 레지스터 할당
4. 명령어 Fetch/Decode	Warp Scheduler	각 warp의 다음 명령어를 가져와 디코딩
5. Warp 스케줄링	Warp Scheduler	실행 준비된 warp 중 하나를 선택하여 명령어 발행
6. 명령어 실행	실행 유닛	선택된 유닛(CUDA Core, Tensor Core, SFU, LD/ST)에서 실행
7. 메모리 접근	LD/ST → 캐시 → VRAM	필요 시 공유메모리/L1/L2/글로벌 메모리 접근
8. 결과 기록	Register File	연산 결과를 스레드 레지스터에 저장
9. 동기화/완료	SM → GPU → CPU	블록 내 <code>__syncthreads()</code> , 커널 완료 시 CPU에 통지

Warp 스케줄링 상세

GPU 성능의 핵심은 warp 스케줄러가 매 사이클 "놓지 않는 것"에 있음.

동작 원리:

- SM에는 동시에 수십 개의 warp가 상주
- 각 warp는 "실행 가능(eligible)" 또는 "대기(stalled)" 상태
- 대기 원인: 메모리 응답 대기, 의존성 대기, 동기화 대기 등

- 스케줄러는 매 사이클 실행 가능한 warp 중 하나를 선택해 명령어 발행
- 한 warp가 메모리 응답을 기다리는 동안 다른 warp가 실행됨 → **지연 은닉(latency hiding)**
 - ⇒ 실행 유닛(CUDA Core 등)이 아무것도 안 하는 시간을 없애는 것이 목적!

상태	의미
Eligible	다음 명령어 실행 준비 완료
Stalled (memory)	글로벌 메모리 응답 대기 중
Stalled (dependency)	이전 명령어 결과 대기 중
Stalled (sync)	<code>__syncthreads()</code> 등 배리어 대기 중

충분한 상주 warp가 있어야 지연 은닉이 작동함 → occupancy가 중요한 이유.

지연 은닉 (Latency Hiding) 도식

시간 →	Cycle 1	Cycle 2	Cycle 3	Cycle 4	Cycle 5	...
Warp A:	[연산]	[메모리요청]	[대기]	[대기]	[결과도착→연산]	
Warp B:	[대기]	[연산]	[연산]	[메모리요청]	[대기]	
Warp C:	[대기]	[대기]	[연산]	[연산]	[연산]	

↑ 스케줄러가 매 사이클 실행 가능한 warp를 선택

실행 유닛이 비는 사이클 없이 계속 일하게 하는 것이 목표.

메모리 접근 흐름

스레드 요청 → Register (hit?) → Shared Memory (명시적)
↓ (miss 또는 글로벌 접근)
L1 Cache (hit?)
↓ (miss)
L2 Cache (hit?)
↓ (miss)
VRAM (글로벌 메모리)

- 공유 메모리는 프로그래머가 명시적으로 사용 (자동 캐시 아님)
- L1/L2는 하드웨어가 자동 관리
- VRAM 접근 시 coalescing 여부에 따라 트랜잭션 수 결정

명령어 파이프라인

GPU도 파이프라인을 사용하지만, CPU와 접근 방식이 다름:

항목	CPU	GPU
파이프라인 깊이	깊음 (10~20+ 단계)	상대적으로 얕음
분기 예측	정교한 예측기 사용	예측 없음, divergence 시 양쪽 실행
버블(빈 슬롯) 대응	예측/재정렬로 최소화	warp 전환으로 채움
Out-of-Order	적극 활용	없거나 매우 제한적 (In-Order 위주)

GPU는 파이프라인 버블을 다른 warp 실행으로 채우기 때문에, 복잡한 예측/재정렬 회로가 불필요.

Branch Divergence 처리 흐름

Warp 내 32스레드가 if-else를 만났을 때:

1. 조건 평가 → 일부 true, 일부 false
2. True 경로 실행 (false 스레드는 비활성 마스크)
3. False 경로 실행 (true 스레드는 비활성 마스크)
4. 합류(reconvergence) → 모든 스레드 다시 활성화

결과: 양쪽 경로를 순차 실행하므로 실질 처리량 절반으로 감소

모든 스레드가 같은 경로를 타면 divergence 없음 → 최대 효율.

커널 실행의 전체 생명주기

단계	설명
Launch	CPU가 <code>kernel<<<grid, block>>>(args)</code> 호출, 명령이 GPU 명령 큐에 삽입
Distribute	GigaThread Engine이 블록을 SM에 분배. 자원 부족한 SM에는 배정 안 함
Execute	SM 내에서 warp 단위로 실행. 완료된 warp는 자원 반환
Retire	블록 내 모든 warp 완료 → 블록 retire → SM에 새 블록 배정 가능
Complete	모든 블록 완료 → 커널 종료 → CPU에 완료 신호

블록은 SM에 한번 배정되면 끝날 때까지 이동하지 않음. 블록 간 실행 순서는 보장되지 않으며, 이것이 GPU의 확장성(scalability)의 핵심 — SM 수가 다른 GPU에서도 같은 커널이 동작.

Occupancy와 성능 관계

Occupancy	의미	영향
높음	SM에 상주 warp 많음	지연 은닉 충분, 실행 유닛 활용률 높음
낮음	SM에 상주 warp 적음	메모리 대기 시 실행 유닛 놀게 됨

Occupancy를 낮추는 요인:

- 스레드당 레지스터 과다 사용
- 블록당 공유 메모리 과다 사용
- 블록 크기가 너무 크거나 작음

단, occupancy가 높다고 항상 성능이 좋은 것은 아님. 캐시 thrashing이나 bank conflict가 발생할 수 있으므로 균형이 중요.

동기화 메커니즘

범위	방법	설명
Warp 내	암묵적 동기 (SIMT)	같은 명령을 함께 실행하므로 별도 동기화 불필요
블록 내	<code>__syncthreads()</code>	블록 내 모든 스레드가 이 지점에 도달할 때까지 대기
블록 간	Atomic + 글로벌 메모리	직접적 배리어 없음. Atomic 연산 또는 커널 분리로 해결
커널 간	암묵적 순서 (같은 스트림)	같은 스트림 내 커널은 순차 실행 보장
GPU-CPU 간	<code>cudaDeviceSynchronize()</code> 등	CPU가 GPU 작업 완료를 대기

비동기 실행과 스트림

GPU는 CPU와 비동기로 동작함:

- CPU가 커널을 발행하면 즉시 반환 (GPU가 백그라운드 실행)
- **스트림(Stream)**: 순서가 보장되는 명령 큐. 서로 다른 스트림은 병렬 실행 가능
- 데이터 전송과 커널 실행을 서로 다른 스트림에 넣으면 오버랩 가능

```
Stream 1: [H→D 전송] [커널 A 실행] [D→H 전송]
Stream 2:           [H→D 전송] [커널 B 실행] [D→H 전송]
           ↑ 전송과 연산이 겹침 → 총 시간 단축
```



GPU 동작의 본질은 "대량의 스레드를 계층적으로 조직하고, warp 단위로 스케줄링하며, 메모리 지연을 스레드 전환으로 숨기는 것"이다. 프로그래머는 블록 크기·메모리 접근 패턴·레지스터 사용량을 조절하여 이 하드웨어 구조에 맞는 코드를 작성해야 최대 성능을 얻을 수 있다.